Artificial Intelligence

Supervised Learning

Classification vs Regression

- Classification: for the example $(x_1, x_2, \dots, x_n)$ predict the label y (e.g., face recognition)
- Regression: for the example $(x_1, x_2, \dots, x_n)$ predict a real number y (e.g., house price prediction)

$\boxed{1.}$ $\boxed{2}$

Real-life Example of Linear Regression

- Predict the price of a house based on its Area, Garage Area, Land Contour, Utilities, etc.
- Predict drug dosage according to blood pressure of the patients.
- Forecasting: ability to forecast trends and make predictions that are feasible.

Linear Regression

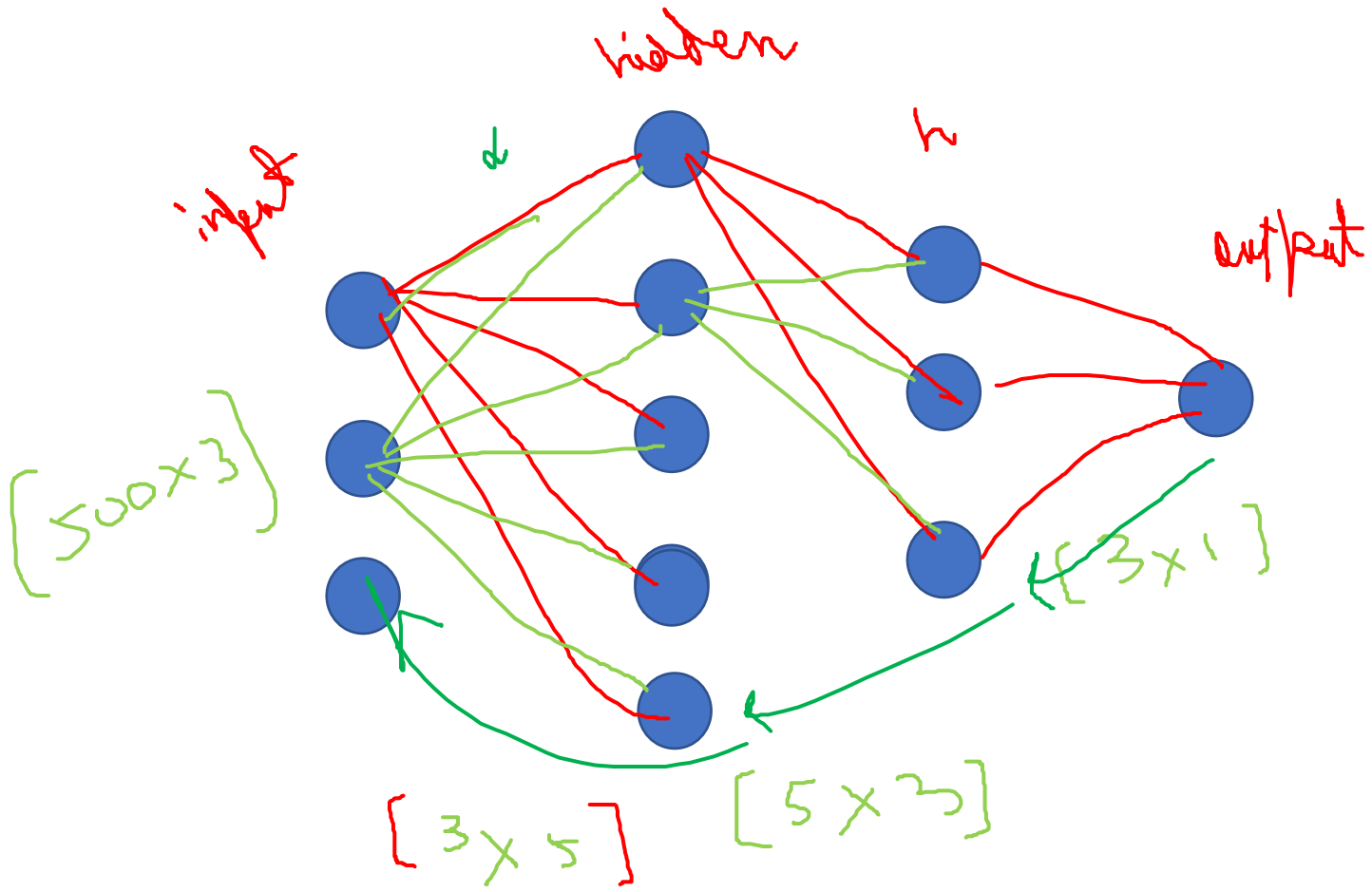
- Model can be evaluated using below methods:

- Mean absolute Error: $\frac{1}{\mathrm{N}}\Sigma(|T-O|)$

- Mean Squared Error: $\frac{1}{\mathrm{N}}\Sigma(T-O)^2$

- Root Mean Squared Error: $\sqrt{\frac{1}{N}\Sigma(T-0)^2}$

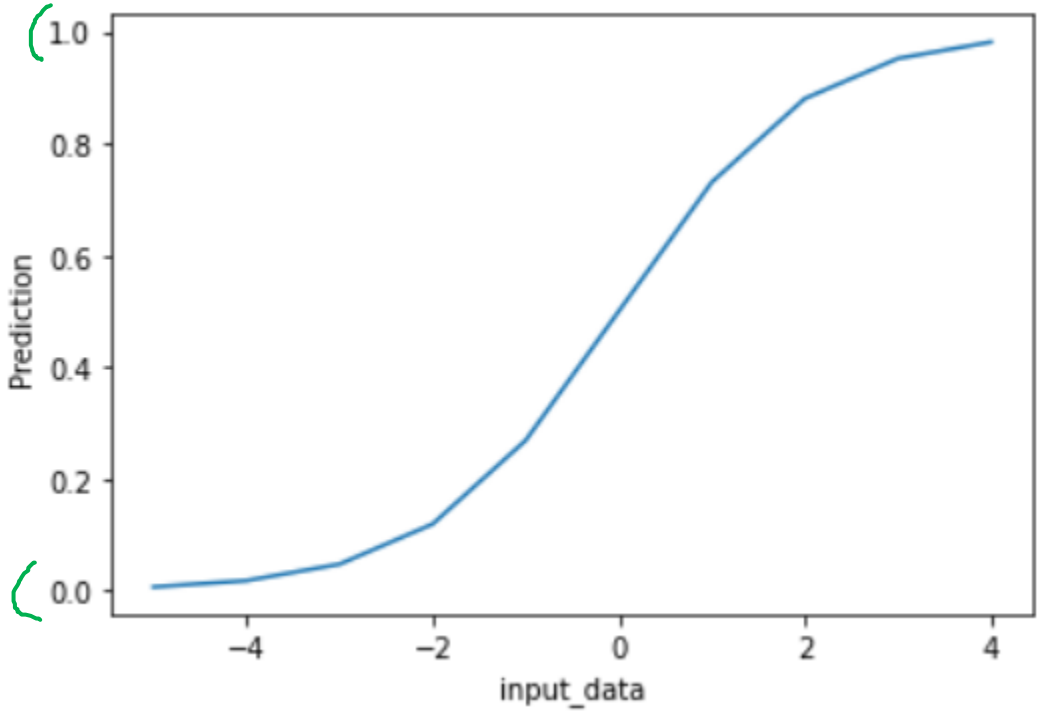
Neural Network: Weight Matrices



Activation Function

- Sigmoid [0,1]
  - Squashing Function

```
from math import exp
from matplotlib import pyplot as plt

def sigmoid(x):
    return 1.0 / (1.0 + exp(-x))
input_data=[]
for data in range(-5, 5):
    input_data.append(data)
prediction=[]
for data in input_data:
    prediction.append(sigmoid(data))
plt.xlabel('input_data')
plt.ylabel('Prediction')
plt.plot(input_data, prediction)
plt.show()
```

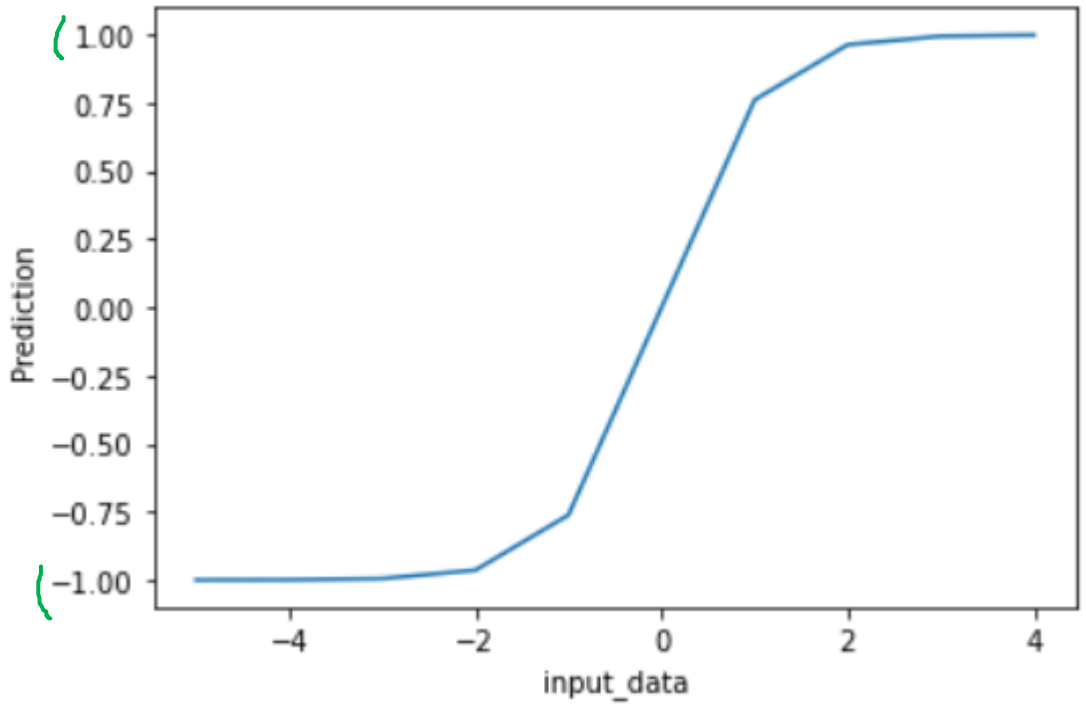


Used in 1990's

Activation Function

- Tanh
  - [-1,+1]

```
from math import exp
from matplotlib import pyplot as plt

def tanh(x):
    return (exp(x) - exp(-x)) / (exp(x) + exp(-x))
input_data=[]
for data in range(-5, 5):
    input_data.append(data)
prediction=[]
for data in input_data:
    prediction.append(tanh(data))
plt.xlabel('input_data')
plt.ylabel('Prediction')
plt.plot(input_data, prediction)
plt.show()
```

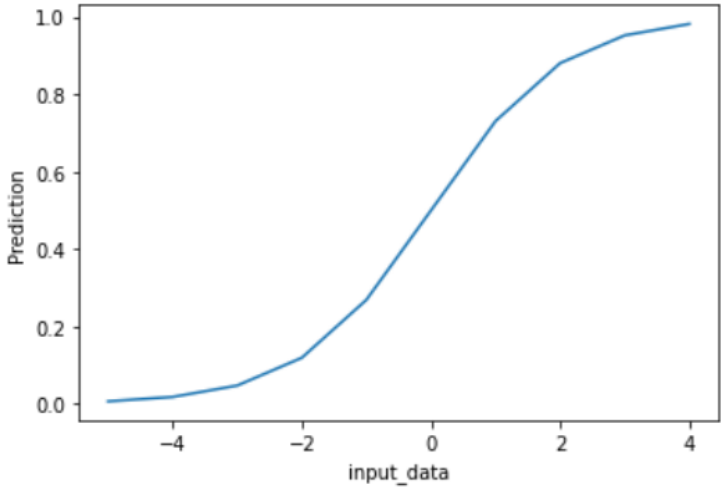


Used till 2010's

Activation Functions

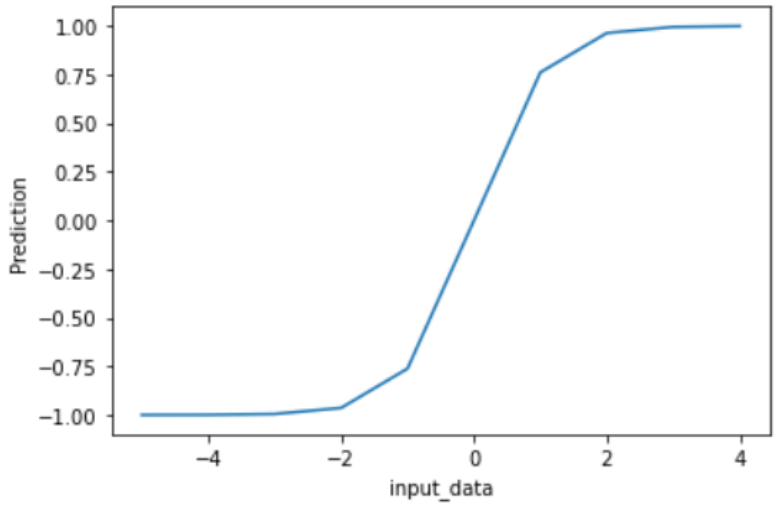
A non-linear activation function applies a non-linear transformation to the input, which means that the **output of the activation function is not proportional to the input.**

- Sigmoid [0,1]
- Tanh [-1,+1]
- ReLU [0.0, max(x)]

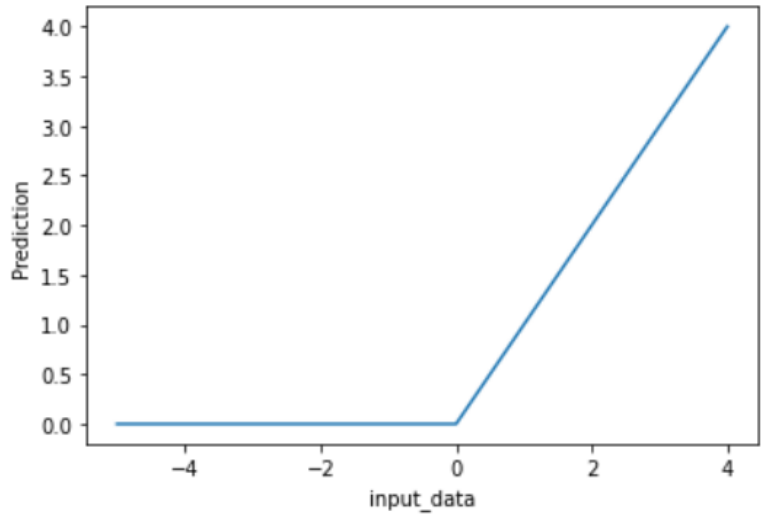


Sigmoid

Tanh



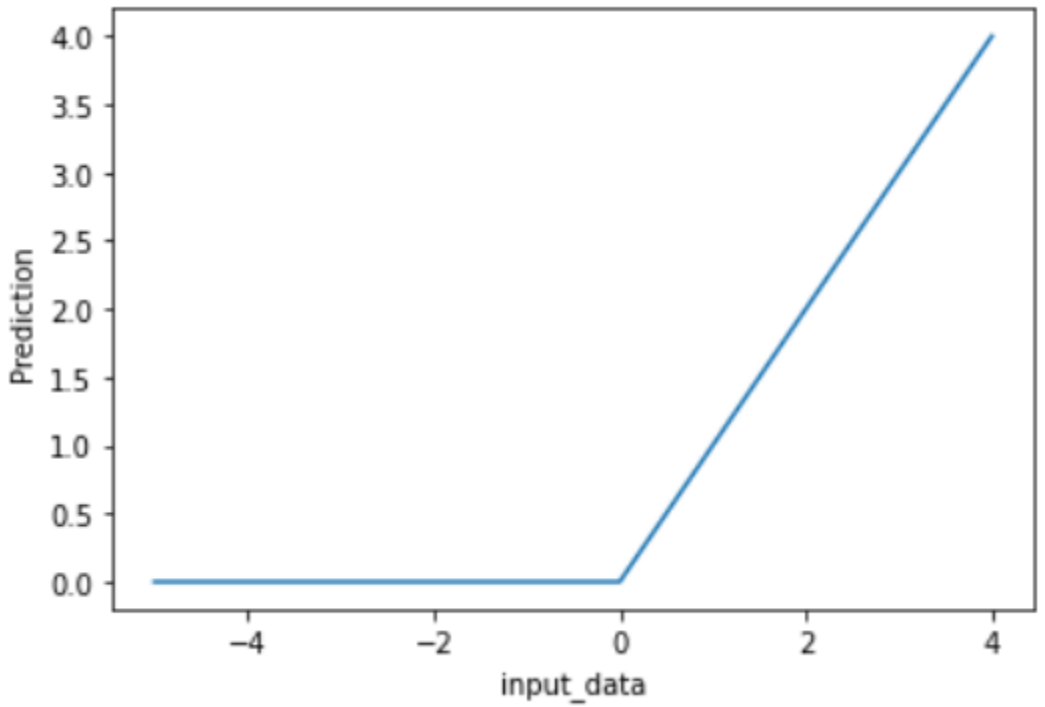
ReLU



Activation Function

- ReLU
  - [0,max(x)]

```
from matplotlib import pyplot as plt

def ReLU(x):
    return max(0.0, x)

input_data=[]
for data in range(-5, 5):
    input_data.append(data)
prediction=[]
for data in input_data:
    prediction.append(ReLU(data))
plt.xlabel('input_data')
plt.ylabel('Prediction')
plt.plot(input_data, prediction)
plt.show()
```



Use for hidden layers

Multiclass Classification: MNIST

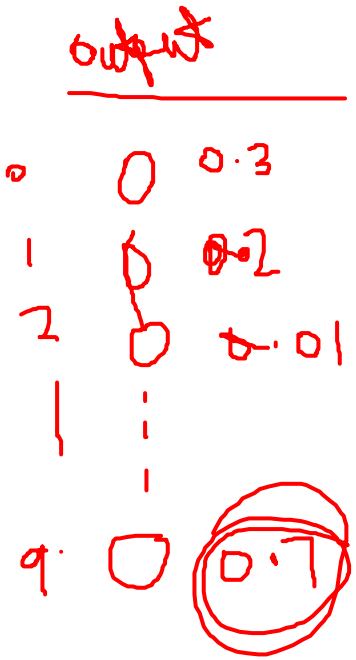
```
import numpy as np
import matplotlib.pyplot as plt

img_size = 28 * 28 # 28x28
labels = 10 #  0-9

train_data = np.loadtxt("C:\\Users\\LENOVO\\Downloads\\mnist_train.csv", delimiter=",")
test_data = np.loadtxt("C:\\Users\\LENOVO\\Downloads\\mnist_test.csv", delimiter=",")
```

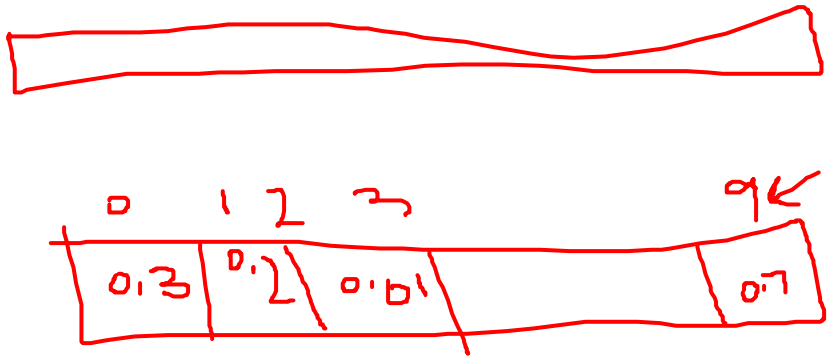
```
print(test_data.shape)
print(train_data.shape)
```

```
(10000, 785)
(60000, 785)
```

```
#Normalize Data
train_imgs = np.asarray(train_data[:, 1:]) * 1/255
test_imgs = np.asarray(test_data[:, 1:]) * 1/255

train_labels = np.asarray(train_data[:, :1])
test_labels = np.asarray(test_data[:, :1])
```

One-hot Encoding



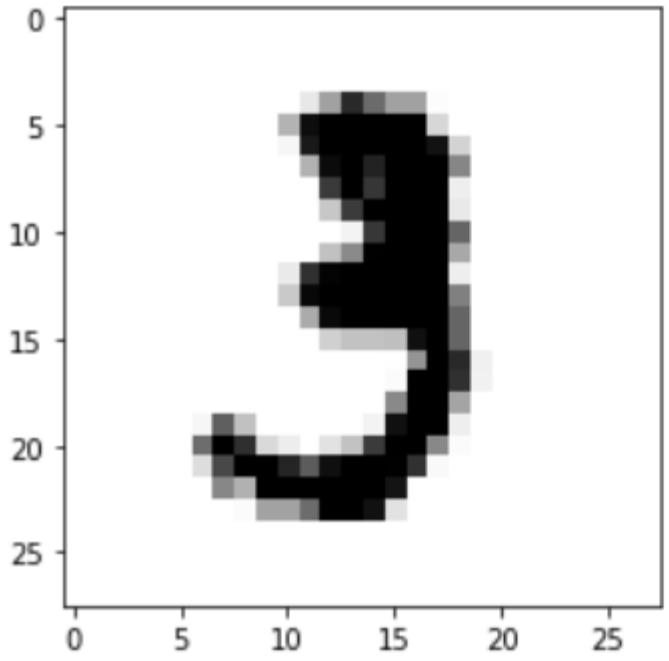


- To make the ground truth labels compatible with the softmax output of the neural network, you one-hot encode these labels.
- This means that you represent each ground truth label as a binary vector with a 1 in the position corresponding to the true class and 0s in all other positions.
- For example, if the ground truth label is “9" in the MNIST dataset, it is one-hot encoded as follows:
- Ground Truth Label "7" → One-Hot Encoding: [0, 0, 0, 0, 0, 0, 0, 0, 0, 1]

Multiclass Classification: MNIST

```
#Image Visualization
img = train_imgs[10].reshape((28,28))
plt.imshow(img,cmap='Greys')
```

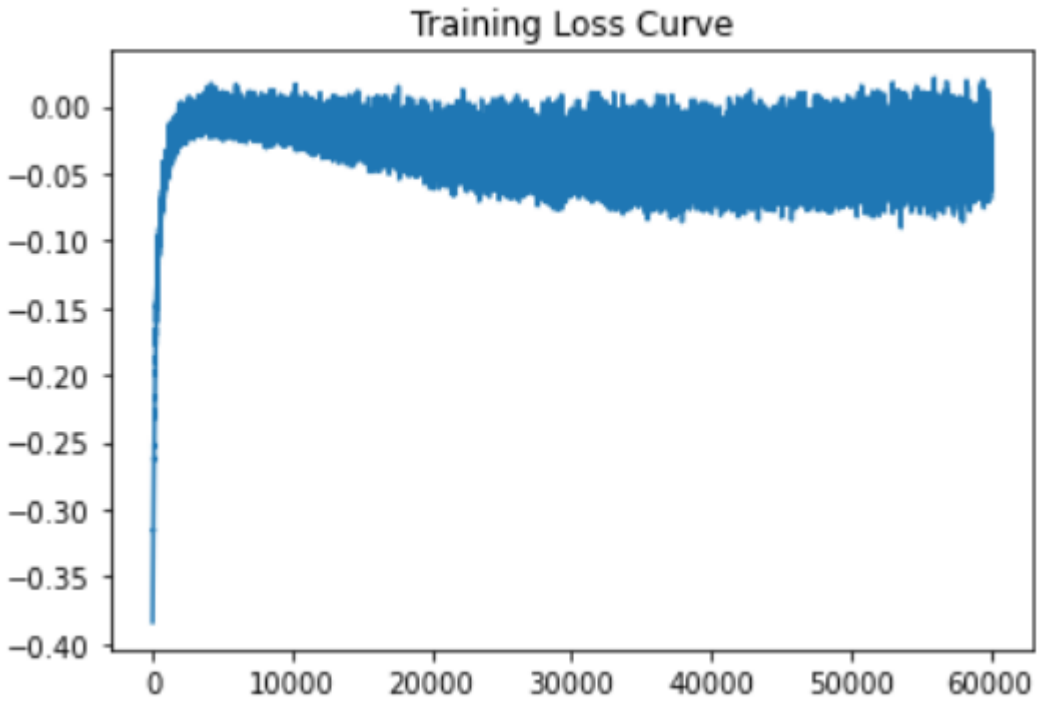
```
<matplotlib.image.AxesImage at 0x22a11441f70>
```




```
train_labels[10]
```

```
array([3.])
```

```python
def train(self, input_vector, target_vector):


    input_vector = np.array(input_vector, ndmin=2).T
    target_vector = np.array(target_vector, ndmin=2).T

    output_hidden = np.dot(self.w_h,
                           input_vector)
    output_hidden = sigmoid(output_hidden)

    output_nw = np.dot(self.w_o, output_hidden)
    output_nw = sigmoid(output_nw)

    err = target_vector - output_nw
    # update the weights:
    tmp = err * output_nw * (1.0 - output_nw)
    tmp = self.lr  * np.dot(tmp, output_hidden.T)
    self.w_o -= tmp


    hidden_err = np.dot(self.w_o.T, err)

    tmp = hidden_err * output_hidden * (1.0 - output_hidden)
    self.w_h -= self.lr * np.dot(tmp, input_vector.T)

    return err
```



Challenges in Training Neural Network

Vanishing Gradient

Network is unable to propagate useful gradient information from the output end of the model back to the layers near the input end of the model which eventually leaves the weights of the initial or lower layers nearly unchanged This means that the updates to the weights in early layers are so small that these layers fail to learn meaningful features from the data.

Exploding Gradient

In contrast, During Backpropagation if the gradients keep on getting larger and larger as the backpropagation algorithm progresses. This, in turn, causes very large weight updates and causes the gradient descent to diverge.

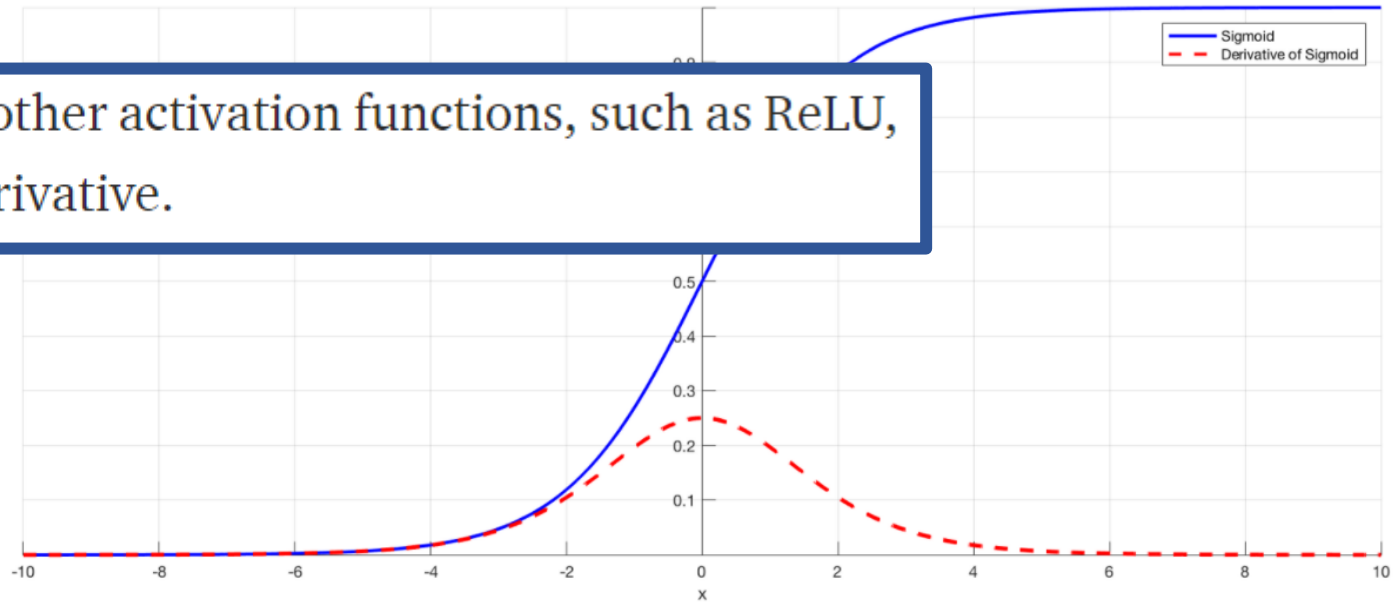
- Solution:
  - Alternate Weight Initialization Methods
  - Alternate Activation Function
  - Gradient clipping

https://www.analyticsvidhya.com/blog/2021/05/how-to-initialize-weights-in-neural-networks/#:~:text=Step%2D1%3A%20Initialization%20of%20Neural,to%20linear%20combination%20(A).

Vanishing Gradient

- The gradients of the loss function approaches zero, making the network hard to train.

The simplest solution is to use other activation functions, such as ReLU, which doesn't cause a small derivative.



How to Identify Vanishing or Exploding Gradient?

- Vanishing
  - The parameters near output layers changes significantly whereas change will become to zero near input layers.
  - Updated weights may become 0 during training.
  - Network learns very slowly, or training stops just after a few iterations.

- Exploding
  - There is an exponential growth in the model parameters.
  - Updated weights may become NaN during training. If the loss becomes too large to be represented as a floating-point number.
  - Their will be a rapid or sudden increase in learning.